

Emotion Detection System

Technical Reference: models, architecture and infrastructure

Input module of the Humanoid Assistant — Final Year Project

Repository: AhmadShazy/Emotion-Detection-System | Contract version 1.2 | 3 September 2026

1. What this module is

This system reads human emotion from four kinds of input and emits a single, stable JSON contract. It is the **input stage** of a three-part project:

Stage	Responsibility	Owner
Input module	Captures nothing itself. Analyses text, audio, video and live streams sent by a browser; fuses the modalities; emits the contract.	This repository
LLM brain	Consumes the contract and decides what to say.	Team member
Avatar output	Renders the reply.	Team member

The defining architectural constraint: the server owns no camera and no microphone. Every byte of audio and video is captured in the user's browser and uploaded. This is what allows many people to use one deployed instance at the same time; a server-side capture device would restrict the whole system to a single user on one machine.

2. Models in use

Five pre-trained models, all running on CPU. None are fine-tuned or retrained at this stage; the current focus is architecture and consistency.

Purpose	Model	Identifier / variant	Size
Text emotion	RoBERTa (go_emotions)	SamLowe/roberta-base-go_emotions	~500 MB
Speech emotion (SER)	SpeechBrain wav2vec2	speechbrain/emotion-recognition-wav2vec2-IEMOCAP	~360 MB
Speech to text — uploads	OpenAI Whisper	base	~140 MB
Speech to text — live call	faster-whisper (CTranslate2)	tiny, int8 quantised	~75 MB
Face expression	MediaPipe FaceLandmarker	face_landmarker.task (float16)	~4 MB

Model output classes

Model	Classes it produces
RoBERTa go_emotions	28 fine-grained labels (admiration, annoyance, remorse, grief, ...), mapped internally to 7 core states
SpeechBrain IEMOCAP	4 classes: happy (hap), angry (ang), neutral (neu), sad (sad)
MediaPipe FaceLandmarker	478 facial landmarks plus 52 blendshape coefficients, classified into expression states from action-unit equivalents

Two deliberate asymmetries. The live call uses faster-whisper *tiny* while uploads use Whisper *base*: measured, *tiny* transcribes in 2.7 s against *base* at 6.6 s, and latency is the product on a live call while accuracy matters more on an upload the user is already waiting for. Similarly, speech emotion reads the last 6 seconds on a live turn but the whole file on an upload — capping upload audio was tested and changed 2 of 10 labels, losing signal in both cases. Both are single call sites and can be revisited without architectural change.

3. Architecture

Four analysis paths converge on one shared component that builds the payload. Because there is exactly one constructor, the four modes cannot structurally drift apart in what they emit.

Mode	Input	Models used	Transport
Text	Typed message	RoBERTa	POST /analyze/text
Voice	Recorded or uploaded WAV	SpeechBrain + Whisper base + RoBERTa	POST /analyze/voice
Video	Recorded or uploaded video file	MediaPipe + SpeechBrain + Whisper base + RoBERTa	POST /analyze/video
Live call	Real-time browser mic + camera	MediaPipe + SpeechBrain + faster-whisper + RoBERTa	WebSocket /ws/stream

Processing chain

```
browser capture -> upload / stream -> decode (ffmpeg) -> [face | speech-emotion | speech-to-text]
-> text emotion
    -> fusion engine -> LLM adapter -> JSON contract
```

The three analysis stages run **concurrently** on every path, not in sequence. They read the same input and share nothing, so ordering never mattered; it was sequential only by accident of how the code was first written. Measured across real recordings, running them in parallel was 13 percent faster with outputs identical in all five test cases.

Key modules

Module	Responsibility
api.py	FastAPI entry point, API-key gate, static frontend, health
src/core/model_registry.py	Loads all models once at startup and shares them read-only
src/core/config.py	The only module that reads environment variables (enforced by a test)
src/video/ingest.py	ffmpeg/ffprobe wrapper: validation, frame extraction, 16 kHz audio
src/streaming/turn_detector.py	Detects speech turns on the live call using an adaptive noise floor
src/streaming/call_session.py	Per-connection live-call state
src/streaming/emotion_state_manager.py	Fusion engine: weighting, temporal smoothing, conflict detection
src/streaming/llm_adapter.py	Builds the final JSON contract
src/streaming/unified_pipeline.py	The single convergence point every mode passes through

4. Fusion engine

Decision-level late fusion. Each modality votes with a weight, and the result is smoothed against recent history. The engine is deliberately pure: it imports only Python's collections module, so it can be run and tested without any model, server or file system.

Modality	Base weight	Trust calibration	Rationale
Voice	0.40	0.90	Hardest to consciously control, so most informative

Modality	Base weight	Trust calibration	Rationale
Face	0.30	0.80	Easily posed; also the noisiest signal
Text	0.30	1.00	Literal content; reliable but easily contradicted by delivery

The effective weight is $\text{base} \times \text{reliability} \times \text{calibration}$, and confidence is the mean confidence of the modalities that agreed, scaled by the share of total weight that agreed.

Conflict detection

The capability that distinguishes this system from a sentiment classifier: detecting when the face and the voice disagree, which is what separates someone who *is* calm from someone who is *hiding* that they are not.

Type	Face	Voice	Interpretation
masked_anger	happy	angry	Smiling through irritation
suppressed_frustration	neutral	angry	Holding a professional expression
masked_sadness	happy	sad	Putting on a brave face
internal_sadness	sad	neutral	Sadness visible despite a level voice

When a conflict is detected the engine re-weights toward the modality it trusts for that case — the voice in three of the four types, the face for internal_sadness.

5. Output contract

Every mode emits the same five top-level keys. Nothing is ever null and no key is ever absent. Full specification in `contract/CONTRACT.md`, with nine frozen example payloads served by a mock endpoint that needs no models.

Key	Contents
<code>session_id</code>	Conversation identifier; keeps emotional history alive across turns
<code>user_input</code>	The user's words (may be an empty string) and a UTC timestamp
<code>emotion_analysis</code>	<code>dominant_emotion</code> , <code>confidence</code> , and probabilities over all 15 classes
<code>tone_analysis</code>	Suggested conversational tone and its confidence
<code>conflict_analysis</code>	<code>detected</code> , <code>type</code> , and a plain-English explanation

The 15 LLM-facing emotion classes

happy sad angry surprised neutral empathetic concerned fear disgust shame
guilt anxiety frustration joy calm

Two things a consumer must know. First, `dominant_emotion` is frequently *not* the highest entry in `emotion_probabilities` — never take the `argmax`. Second, a detected conflict is actionable at *any* confidence: confidence measures how much the modalities agreed, and a conflict is precisely the case where they do not, so those payloads score low by construction. Branch on `conflict_analysis.detected` before reading the confidence thresholds.

6. API surface

Endpoint	Method	Purpose	Auth
<code>/analyze/text</code>	POST	Text emotion analysis	Key
<code>/analyze/voice</code>	POST	WAV upload; normalised to 16 kHz mono	Key
<code>/analyze/video</code>	POST	Video upload; frames plus audio	Key
<code>/ws/stream</code>	WS	Live call; binary audio and video frames	Key
<code>/mock/*</code>	GET/POST	Frozen contract examples; loads no models	Key
<code>/health</code>	GET	Model status for platform probes	Public
<code>/</code>	GET	Browser front end	Public

Authentication is a shared key sent as an `x-API-Key` header, or as `?api_key=` for the WebSocket, since browsers cannot set headers on a WebSocket handshake. There is deliberately **no exemption for local requests**: an earlier bypass for loopback callers meant the authentication path was the one path local testing never exercised.

Live-call wire protocol

Frame	Prefix	Payload
Audio	0x01	16 kHz signed 16-bit PCM, 2048-sample frames from an <code>AudioWorklet</code>
Video	0x02	JPEG frames at a server-announced rate (default 3 fps)
Control	JSON text	Handshake, stop, and typed status or error codes

7. Operational parameters

Parameter	Value	Meaning
AUDIO_SAMPLE_RATE	16000 Hz	Required by wav2vec2; enforced on every path
MAX_VIDEO_SECONDS	30 s	Upload analysis window, trimmed before decoding
FRAME_SAMPLE_FPS	5 fps	Frames sampled from uploaded video
LIVE_VIDEO_FPS	3 fps	Frame rate the server asks the browser for
MAX_TURN_SECONDS	20 s	Cap on a single live-call speaking turn
TRAILING_SILENCE_SECONDS	0.9 s	Silence that closes a turn
MIN_ABSOLUTE_THRESHOLD	0.002 RMS	Shared definition of silence
SER_MAX_SECONDS	6.0 s	Audio the live path sends to speech emotion
MAX_ACTIVE_CALLS	2	Concurrent live calls per process
INFERENCE_WORKERS	2	Analysis thread pool size
MAX_FRAME_BYTES	512 KB	Largest accepted video frame
SESSION_TIMEOUT_MINUTES	30	Idle session expiry

Turn detection uses an **adaptive noise floor** with hysteresis rather than a fixed threshold, so it works for quiet speakers and noisy rooms without tuning. Thread pools are deliberately small: measured on four cores, more concurrent model instances was slower, not faster.

8. Infrastructure and deployment

Aspect	Detail
Runtime	Python 3.10, FastAPI, uvicorn (single worker)
System dependencies	ffmpeg and fprobe; OpenGL and PortAudio shared libraries
Dependency pinning	requirements.lock, 94 packages pinned from the environment the tests pass in
Container	python:3.10-slim; all five models baked into the image
Image size	Approximately 5 GB; measured resident memory 1.54 GB
CPU build	torch and torchaudio installed from the CPU wheel index
Build pipelines	GitHub Actions to GHCR, and Cloud Build to Artifact Registry
Current image	linux/amd64, digest sha256:6b569e9c..., verified in CI

Why models are baked into the image

This system fails *silently*: the model registry catches every loading exception and the health endpoint reports success regardless. An instance missing a model therefore does not crash — it starts cleanly, passes its health check, and returns neutral for every request forever. Downloading models at boot makes that outcome depend on network conditions, so they are shipped inside the image instead.

Both pipelines therefore run an explicit verification step: the built image is started with **no network access at all** and must report every model loaded, ffmpeg present, the MediaPipe model file present, torch as a CPU build, and the server in full mode. This has passed on the current image.

Deployment requirements

Requirement	Value	Why
Memory	4 GB minimum	1.54 GB resident; 2 GB tiers do not fit
Architecture	linux/amd64	Current image is x86-64; ARM needs a rebuild
HTTPS	Required	Browsers expose camera and microphone only in a secure context; over plain HTTP the voice, video and live-call modes do not work at all
Instances	One	Session state and the call counter are per-process
Request timeout	Raise from default	Platform request timeouts apply to WebSockets and would cut a live call short

9. Testing and quality controls

86 automated tests. Most need no models and run in well under a second.

Suite	Covers
test_contract.py	Payload shape, all 15 classes present, probabilities summing to 1, plus architecture rules enforced as tests
test_fusion.py	Fusion engine directly: all four conflict types, confidence calibration, persistence behaviour
test_auth.py	API-key gate for HTTP and WebSocket, and that no header can substitute for a key
test_live_stream.py	Turn detection across loudness ranges, packet sizes and edge cases; a real WebSocket round trip
test_video_upload.py	Upload validation and decoding

Architecture rules enforced by tests

Several design rules are checked automatically rather than by convention, so breaking one fails the build:

- No module outside config.py may read environment variables.
- The speech-emotion reliability formula must exist in exactly one place.
- The silence threshold must be shared between the upload and live paths.
- The SpeechBrain compatibility patches must exist in exactly one place.

These exist because duplication has already caused real failures here. The SpeechBrain patches were once written out in four places, two of them incomplete, and the incomplete copies were the ones the container build depended on — the build failed while the running server loaded the same model without complaint.

10. Status and open items

Item	State
Text, Voice, Video, Live Call	Working and tested end to end
Output contract	Frozen at version 1.2; examples available via /mock
Container image	Built and verified in CI with no network access
Public deployment	Not yet live — host selection in progress
Model accuracy and retraining	Deliberately deferred; current focus is architecture and consistency

Known open decisions

- Speech-to-text model differs between the live call and uploads (latency against accuracy). Both are single call sites.
- Speech emotion reads 6 seconds on the live call but the whole file on uploads.
- The dependency lock was produced on Windows and cannot pin two Linux-only transitive packages; regenerating it inside the built image would close that gap.
- Delivery to the LLM stage is currently implemented on one endpoint of four; the other three return the payload in the response instead.

A fuller engineering backlog, ordered by priority with effort estimates, is maintained in `BACKLOG.md` in the repository.